

FAIRWORK: A DECENTRALIZED FREELANCE MARKETPLACE WITH SMART CONTRACT ESCROW AND DISPUTE RESOLUTION

A PROJECT REPORT SUBMITTED TO SRM INSTITUTE OF SCIENCE & TECHNOLOGY
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF
MASTER OF COMPUTER APPLICATIONS

SUBMITTED BY:

VIGNESH V

REG NO: RA2532241040045

UNDER THE GUIDANCE OF

Dr. M. Sivasakthi, M.Sc., M.Phil., Ph.D., NET

Associate Professor, Department of Computer Applications



DEPARTMENT OF COMPUTER APPLICATIONS (MCA)
FACULTY OF LIBERAL ARTS AND BUSINESS STUDIES
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
VADAPALANI CAMPUS, CHENNAI – 600026
ACADEMIC YEAR: 2025 – 2026

DEPARTMENT OF COMPUTER APPLICATIONS (MCA)
FACULTY OF LIBERAL ARTS AND BUSINESS STUDIES
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
VADAPALANI CAMPUS, CHENNAI – 600026



BONAFIDE CERTIFICATE

This is to certify that the project report titled **“FAIRWORK: A DECENTRALIZED FREELANCE MARKETPLACE WITH SMART CONTRACT ESCROW AND DISPUTE RESOLUTION”** is the bonafide work done and submitted by **VIGNESH V (Reg. No: RA2532241040045)** during the **2025–2026** academic year, in partial fulfillment of the requirements for the award of the degree of **MASTER OF COMPUTER APPLICATIONS**, at **SRM INSTITUTE OF SCIENCE & TECHNOLOGY, Vadapalani, Chennai**.

Signature of the Guide

Dr. M. Sivasakthi, M.Sc., M.Phil., Ph.D., NET
Associate Professor, Department of Computer
Applications
SRM IST, Vadapalani Campus

Signature of the HOD

Dr. J. Anitha Ruth, M.S., Ph.D.
Head of the Department
Department of Computer Applications
SRM IST, Vadapalani Campus

Submitted for Project Work Viva-Voce Examination held on: _____

Place: VADAPALANI, CHENNAI

Date: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, I would like to express with a deep sense of gratitude my heartfelt thanks to the management of **SRM Institute of Science & Technology** for providing modern laboratory infrastructure, technical facilities, and an encouraging academic environment.

I wish to express my sincere gratitude to the **Dean**, Faculty of Liberal Arts and Business Studies, for his constant administrative support and encouragement throughout the course of this MCA program.

I express my deepest gratitude to my esteemed project guide, **Dr. M. Sivasakthi, M.Sc., M.Phil., Ph.D., NET**, Associate Professor, Department of Computer Applications, for her thoughtful guidance, insightful feedback, technical suggestions, and patient mentoring across every phase of designing and implementing this major project.

I extend my sincere thanks to **Dr. J. Anitha Ruth, M.S., Ph.D.**, Head of the Department of Computer Applications, for her dedicated academic leadership, departmental support, and for facilitating the computational resources required to successfully complete this project.

I would also like to thank my class coordinator, faculty members, and staff of the Department of Computer Applications for their academic assistance, cooperation, and words of encouragement.

Finally, I convey my warmest gratitude to my parents and friends for their enduring patience, understanding, moral encouragement, and unconditional support during my project endeavors.

VIGNESH V

(Reg. No: RA2532241040045)

Department of Computer Applications (MCA)

SRM Institute of Science & Technology, Vadapalani

ABSTRACT

Decentralized Web3 applications frequently fail to achieve mainstream adoption due to fragmented, confusing user experiences, cryptic blockchain error messages, complex wallet interaction dialogs, and clunky interfaces. For a freelance escrow platform to succeed, it must deliver the visual precision, accessibility, and frictionless responsiveness of modern enterprise consumer software while seamlessly interacting with decentralized smart contracts and backend REST APIs.

This project documentation details the **Frontend Engineering, UI/UX Architecture, React/Vite Client Implementation, Tailwind CSS Design System, Multi-Role User Workflows, and REST/Web3 API Integrations** designed and implemented by **VIGNESH V (Reg. No: RA2532241040045)**.

The frontend application is constructed using a high-performance **React 19, TypeScript, and Vite 6** technical stack styled via a custom **Tailwind CSS** variable token system. Key subsystems engineered include:

1. **Public Landing Page & Discovery Studio:** Engineered a modular, authentic landing experience featuring HeroSection (universal search and interactive escrow simulator), EscrowFlowBlueprint (4-stage visual settlement pipeline), PlatformFeatures (core architectural safeguards), ProjectCalculator (interactive budget and milestone estimator), and VerifiedProtocolShowcase (live Sepolia smart contract registry with real Etherscan links).
2. **Project Detail Hub & Multi-Tab Workspace:** Engineered a collaborative workspace consolidating project overview, proposal evaluation, AI-generated legal contract signing, milestone submission with drag-and-drop file attachments, on-demand GitHub PR CI/CD status inspection (✓ CI Green, ✗ Failing, ◻ In Progress), on-chain escrow funding triggers, and milestone release transactions.
3. **AI Dispute Mediation Interface:** Designed an intuitive dispute resolution portal displaying Gemini AI's objective recommendations, neutral rationale breakdowns, a live 48-hour mutual consensus window countdown, and one-click bilateral acceptance controls.
4. **On-Chain Reputation Modal & Freelancer Showcase:** Implemented an accessible post-completion rating dialog that writes immutable 1–5 star scores to the Sepolia ReputationContract accumulator with real-time MetaMask transaction feedback.
5. **Robust API Client & State Layer:** Developed type-safe REST consumers (projectsApi.ts, disputesApi.ts, contractsApi.ts) with normalized error handling, session token management, and Viem/MetaMask Web3 interaction wrappers (web3.ts).

The application compiles with **zero TypeScript errors (tsc -b && vite build)**, satisfies strict linting rules, and exhibits exceptional performance across mobile, tablet, and desktop viewports, demonstrating how decentralized freelance commerce can be delivered with enterprise-grade usability.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF TABLES	vi
	LIST OF FIGURES	vii
1	INTRODUCTION	1
2	MODULE DESCRIPTION & SYSTEM REQUIREMENTS	5
3	SYSTEM DESIGN & UI/UX ARCHITECTURE	14
4	IMPLEMENTATION & CODE METHODOLOGY	22
5	TESTING, USABILITY & PERFORMANCE VERIFICATION	40
6	CONCLUSION & FUTURE ENHANCEMENTS	48
	APPENDIX: ROUTE MAP & API TYPINGS	50
	REFERENCES	53

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Reusable Design System Component Specification	7
2.2	Frontend Software Stack & Dependency Matrix	13
3.1	Application Route Mapping & Access Authority	21
5.1	Client Form Validation & Security Test Matrix	43
5.2	Web3 Transaction Error Recovery & Fallback Matrix	44
5.3	Google Lighthouse Performance & Accessibility Scores	46

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	React 19 Frontend Component Architecture & State Store Hierarchy	15
3.2	Client Milestone Escrow Lifecycle & Funding Flowchart	17
3.3	Freelancer Work Submission & GitHub CI Telemetry Sequence	18
3.4	AI Dispute Mediation Consensus & Bilateral Acceptance Workflow	19
3.5	Frontend Level 1 Data Flow Diagram (State & Web3 Interaction)	20
4.1	FairWork Public Discovery Studio & Interactive Milestone Escrow Simulator	35
4.2	FairWork Projects Marketplace with Real-Time Filtering & Milestone Budgets	36
4.3	Responsive Project Detail Hub with Multi-Tab Milestone & Contract Management	37
4.4	Project Creation Workspace with Web3 Client Wallet Connection Guardrail	38
4.5	FairWork Ask AI Streaming Assistant Interface	39

CHAPTER 1: INTRODUCTION

1.1 ABOUT THE PROJECT

As the global software engineering workforce shifts toward distributed freelancing and remote technical consulting, peer-to-peer economic tools must keep pace. While blockchain technology provides the decentralized foundation for non-custodial milestone escrow, the ultimate success of such financial protocols hinges upon human adoption. If users are confronted with cryptic cryptographic hashes, unintelligible RPC error codes, disjointed multi-page flows, and complex wallet approvals, they will inevitably retreat to predatory centralized intermediaries.

FairWork addresses this critical adoption barrier by delivering an enterprise-grade, responsive Web3 freelance application built using **React 19, TypeScript, Vite 6, and Tailwind CSS**. FairWork bridges the technical gap between decentralized Ethereum smart contracts and consumer-friendly marketplace software:

- It translates complex Solidity state machines into intuitive visual interfaces (such as interactive escrow ledgers, milestone progress bars, and countdown challenge timers).
- It consolidates the freelance collaboration journey into clear, purpose-built workspaces: project discovery, proposal review, AI-generated legal contract signing, milestone deliverable inspection with automated GitHub CI/CD badges, and one-click wallet payouts.
- It provides non-blocking, asynchronous feedback for Web3 transactions, ensuring users always understand the exact stage of their transaction on the Ethereum Sepolia network.

1.2 PROBLEM STATEMENT & UI/UX MOTIVATION

Traditional Web3 decentralized applications (dApps) suffer from three severe user experience deficiencies:

1. Intimidating Crypto Complexity:

Most dApps present raw contract addresses, hex-encoded transaction data, and unhandled MetaMask rejection messages. Users are left uncertain whether their funds are securely locked in escrow, pending network confirmation, or permanently lost.

1. Fragmented Collaboration Workflows:

In typical freelance engagements, communication occurs in Slack, code is hosted on GitHub, contracts are signed in DocuSign, and payments are wired via PayPal. This fragmentation creates severe misunderstandings regarding milestone completion and makes dispute resolution chaotic and subjective.

1. Inauthentic, Unvetted Marketplace Interfaces:

Many freelance templates rely on fabricated stock personas, fake testimonials, and canned gig cards that misrepresent the decentralized platform as an e-commerce shop. A truly decentralized

freelance marketplace requires an authentic interface that displays verified on-chain smart contracts, real pull request test results, and verifiable blockchain telemetry.

To resolve these challenges, FairWork provides a unified, transparent user interface that abstracts away Web3 complexity while preserving absolute cryptographic auditability.

1.3 SCOPE OF INDIVIDUAL CONTRIBUTION

Within the FairWork project, **VIGNESH V (Reg. No: RA2532241040045)** assumed complete ownership and engineering responsibility for the **Frontend Architecture, UI/UX Design System, React/Vite Client Implementation, Multi-Role User Workflows, and REST/Web3 API Integrations**.

Specifically, the individual scope of work encompassed:

- **Design System Architecture & Primitives:** Constructing an accessible, responsive Tailwind CSS component library featuring custom design tokens, dark/light theme switching, and accessible primitives (Button, Card, Modal, Tabs, Badge, Progress, DataTable, Toast).
- **Public Discovery & Escrow Studio (LandingPage.tsx):** Designing and building HeroSection (with real-time keyword search and interactive escrow ledger simulation), EscrowFlowBlueprint (4-stage settlement pipeline), PlatformFeatures (core architectural safeguards), ProjectCalculator (interactive milestone estimator), and VerifiedProtocolShowcase (Sepolia contract registry).
- **Project Detail Collaboration Workspace (ProjectDetailPage.tsx):** Architecting the centralized multi-tab hub consolidating project overview, applicant proposals, AI-generated legal agreement execution with digital signatures, milestone deliverable submissions, and client escrow actions.
- **GitHub PR CI/CD Badging System:** Designing and integrating the milestone deliverable review UI displaying real-time GitHub Actions test status (✓ CI Green, X CI Failing, 🔄 In Progress, 🔒 Private) with on-demand CI refresh triggers.
- **AI Dispute Mediation Interface (DisputesPage.tsx):** Engineering the dispute resolution portal featuring Gemini AI recommendation cards, neutral technical rationales, a 48-hour mutual consent countdown window, and dual-acceptance controls.
- **On-Chain Reputation Modal (OnChainRatingModal.tsx):** Designing an interactive 1–5 star rating dialog that writes immutable scores to the Sepolia ReputationContract accumulator with dual-storage persistence and MetaMask transaction tracking.
- **Web3 Wallet Client & API Integration (web3.ts, projectsApi.ts):** Implementing type-safe REST API consumers, Viem contract callers, EIP-712 wallet binding interfaces, and session state management.
- **Testing & Usability Auditing:** Ensuring zero TypeScript compilation errors (`tsc -b && vite build`), verifying cross-device responsiveness, and conducting Lighthouse accessibility audits.

1.4 ORGANIZATION OF THE PROJECT REPORT

This report is organized into six chapters:

- **Chapter 1 (Introduction):** Outlines project background, UX challenges, and individual contribution scope.
- **Chapter 2 (Module Description & System Requirements):** Explains the six core frontend modules, design components, hardware, and software requirements.
- **Chapter 3 (System Design & UI/UX Architecture):** Details component trees, user workflow diagrams, Level 0 and Level 1 DFDs, and wireframe layouts.
- **Chapter 4 (Implementation & Code Methodology):** Provides in-depth code analyses of the design primitives, landing components, project detail hub, GitHub CI badge, dispute portal, and rating modal.
- **Chapter 5 (Testing, Usability & Performance Verification):** Reviews TypeScript build validation, responsive viewport tests, form validations, Web3 error handling, and Lighthouse scores.
- **Chapter 6 (Conclusion & Future Enhancements):** Summarizes individual achievements and outlines the frontend roadmap.
- **Appendix & References:** Supplies route definitions, API schemas, and IEEE references.

CHAPTER 2: MODULE DESCRIPTION & SYSTEM REQUIREMENTS

2.1 MODULE 1: DESIGN SYSTEM & REUSABLE ACCESSIBLE PRIMITIVES

To ensure consistent typography, spatial rhythm, and responsive behavior across dozens of application screens, the frontend is constructed upon an atomic design system implemented in Tailwind CSS.

Functional Specifications:

1. Design Tokens & Theme System:

Implemented semantic CSS variables in `index.css` defining background colors (`--bg-base`, `--bg-surface`, `--bg-elevated`), borders (`--border`, `--border-strong`), foregrounds (`--text-foreground`, `--text-muted`, `--text-subtle`), and primary branding colors (`--primary`, `--primary-hover`). A custom `ThemeToggle` component dynamically toggles dark and light themes by mutating the `data-theme` attribute on the HTML document root.

1. Accessible Button Primitive (`Button.tsx`):

Encapsulates standard HTML button semantics while supporting five visual variants (`primary`, `secondary`, `outline`, `ghost`, `danger`), three sizing scales (`sm`, `md`, `lg`), integrated loading spinners (`loading={true}`), and left/right icon slots. Disabled states automatically apply `cursor-not-allowed` and reduce opacity.

1. Card & CardContent Primitives (`Card.tsx`):

Provides elevated surface wrappers with rounded borders (`rounded-2xl`), subtle shadows, and responsive padding.

1. Interactive Tab Controller (`Tabs.tsx`):

Implements accessible tabbed navigation supporting active indicators, count badges, and keyboard navigation.

1. Toast Notification System (`Toast.tsx`):

Constructed a React Context provider (`ToastProvider`) that renders floating notification banners portal-rendered into the top-right viewport. Supports four semantic tones (`success`, `error`, `warning`, `info`) with auto-dismissal timers (default 4500ms).

Table 2.1: Reusable Design System Component Specification

Component Name	File Path	Supported Props / Variants	Primary Role

Button	components/ui/Button.tsx	primary, secondary, outline, ghost, danger, loading	Primary interactive trigger across all forms
Card	components/ui/Card.tsx	className, children, hover	Visual content container with theme borders
Tabs	components/ui/Tabs.tsx	items, value, onChange	Multi-view navigation inside project hubs
Badge	components/ui/Badge.tsx	neutral, success, warning, danger, primary	Status tags and categorical indicators
Progress	components/ui/Progress.tsx	value (0-100), tone	Visual milestone completion tracker
Toast	components/ui/Toast.tsx	title, description, tone, duration	Non-blocking transaction alerts

2.2 MODULE 2: HIGH-CONVERSION PUBLIC DISCOVERY & ESCROW STUDIO

The landing page (`LandingPage.tsx`) acts as the public front door of FairWork. Rather than using generic templates filled with fake stock photos, it provides an authentic, interactive learning and discovery studio.

Functional Specifications:

1. `HeroSection.tsx`:

Features a monumental headline, a universal search bar that routes queries to `/projects?category=...&search=...`, and the **Interactive Escrow Studio**. The studio provides a live ledger simulator showing the exact status of milestone vaults, secured balances, 0% platform deductions, and verified Sepolia smart contract addresses (`0xc0d1...376c`).

1. `EscrowFlowBlueprint.tsx`:

Presents a 4-stage visual pipeline (`01 Scope`, `02 Deposit`, `03 Verify`, `04 Settle`) complete with real Solidity code snippets from `EscrowContract.sol` and verification telemetry.

1. `PlatformFeatures.tsx`:

Replaced all mock freelance gig cards with an architectural showcase of FairWork's core safeguards: Milestone Escrow, AI Dispute Mediation, On-Chain Reputation, and GitHub CI Verification.

1. **ProjectCalculator.tsx:**

An interactive scoping tool allowing clients to choose technical disciplines (Web App, Smart Contract, UI/UX, AI, Mobile) and complexity tiers (Standard, Extended, Production) to calculate estimated milestone counts, turnaround days, and escrow deposits, with direct links into `/projects/new`.

1. **VerifiedProtocolShowcase.tsx:**

Displays the official Sepolia Smart Contract Registry with direct Etherscan links for `EscrowContract.sol`, `DisputeContract.sol`, `ReputationContract.sol`, and `MockUSDC.sol`, alongside the 6-stage project lifecycle.

2.3 MODULE 3: PROJECT DETAIL WORKSPACE & MILESTONE EXECUTION HUB (**ProjectDetailPage.tsx**)

`ProjectDetailPage.tsx` is the central operational command center for project collaboration. It consolidates all project actions into a clean, multi-tab layout.

Functional Specifications:

1. **Multi-Tab Layout (Tabs):**

Organizes project information across six specialized views:

- **Overview:** Project title, budget, timeline countdowns, client and freelancer identity cards with verified wallet addresses, and visual milestone progress bars.
- **Applications:** (Client-only) Review submitted proposals, view freelancer rating metrics, open detailed profile modals, and trigger the hiring action.
- **Contract:** Displays the AI-generated Freelance Services Agreement, including formal terms, budget schedules, and electronic signature panels for client and freelancer.
- **Milestones:** Lists all project milestones with individual budget allocations, delivery deadlines, revision notes, approval triggers, and client-gated release buttons.
- **Files & Deliverables:** Comprehensive repository separating freelancer work submissions from client reference specifications.
- **Activity:** Real-time chronological audit trail of escrow events.

1. **On-Chain Escrow Funding Trigger:**

Provides client-side wallet checks: when a client hires a freelancer, the "**Fund Escrow**" button prompts MetaMask to approve USDC allowance and call `fundEscrow()`, updating UI state with live progress indicators.

1. **Refund Timelock Controls:**

If a client requests a refund, the interface visualizes the active 48-hour challenge timelock with options to **Cancel Request** or **Claim Refund** once the lock expires.

2.4 MODULE 4: GITHUB PR CI/CD TELEMETRY & BADGING INTERFACE

To empower clients to evaluate software deliverables objectively, the deliverable submission and review interfaces integrate live GitHub Continuous Integration (CI) telemetry.

Functional Specifications:

1. Pull Request Submission Interface (`SubmitWorkModal`):

When a freelancer clicks "**Submit Work**" on a milestone, the modal provides a designated input for the GitHub Pull Request URL (`https://github.com/owner/repo/pull/12`) alongside deliverable file attachments and submission notes.

1. Visual CI Status Badges:

In both the Milestones list and the Files tab, deliverables attached to a GitHub PR render a dedicated CI status pill:

- ✓ CI Green (14/14 checks passed): Rendered in emerald green when all check runs succeed.
- ✗ CI Failing: Rendered in rose/danger red when tests fail.
- 🔄 CI In Progress: Rendered in amber/warning yellow when pipelines are actively building.
- 🔒 CI Private / Unavailable: Rendered in subtle neutral when repository access is restricted.

1. On-Demand CI Refresh Trigger (`handleRefreshCI`):

Equips clients with a "**Re-check CI**" button next to each deliverable. Clicking triggers `refreshDeliverableCI()`, polling the GitHub API and updating the badge without requiring full page reloads.

2.5 MODULE 5: AI DISPUTE MEDIATION & CONSENSUS DASHBOARD

(`DisputesPage.tsx`)

When project expectations diverge and a dispute is opened on-chain, `DisputesPage.tsx` provides an impartial, transparent resolution interface.

Functional Specifications:

1. Dispute Overview & Financial Freeze Indicator:

Displays active project metadata, frozen escrow balances, dispute initiator, and reason, with clear warning banners confirming that milestone releases are locked on-chain.

1. AI Assessment Trigger:

Enables either party to request an automated evaluation by clicking "**Request AI Assessment**", which dispatches project deliverables and evidence to the Gemini AI API.

1. **Binary Recommendation Card:**

Visualizes the AI verdict via prominent outcome badges:

- AI Favors Client (Full Refund) Or AI Favors Freelancer (Full Payout).
- Comprehensive written rationale analyzing the technical evidence against agreed milestone criteria.

1. **48-Hour Mutual Consent Window:**

Renders a countdown timer tracking the mutual agreement deadline (`Date.now() + 48h`).

1. **Bilateral Consent Tracker & One-Click Acceptance:**

Displays independent checkboxes indicating whether the Client and Freelancer have accepted the verdict. When both parties consent, the backend relay automatically executes `resolveByArbitrator()` on Sepolia, updating the UI with the mined transaction hash.

2.6 MODULE 6: ON-CHAIN REPUTATION RATING & PROFILE ANALYTICS (`OnChainRatingModal.tsx`, `ProfilePage.tsx`)

To ensure ratings are permanent and portable, the frontend integrates directly with the deployed `ReputationContract.sol`.

Functional Specifications:

1. `OnChainRatingModal.tsx`:

Upon final milestone completion, an interactive modal prompts the user to rate their counterparty:

- Interactive 1–5 star rating selector with hover states and text feedback.
- Testimonial text area (up to 1000 characters).
- Sepolia Blockchain Toggle: Enables the user to submit an on-chain transaction directly to `ReputationContract.submitRating()`.
- Non-blocking error handling: If the user rejects the MetaMask transaction, the modal gracefully falls back to saving the review in MongoDB without crashing.

1. `ProfilePage.tsx` & **Developer Analytics:**

Renders a comprehensive freelancer profile:

- On-chain reputation badge displaying the live Sepolia score accumulator (`totalScore / ratingCount`) with a direct Etherscan link.
- GitHub 52-week contribution heatmap displaying yearly contributions, active commit streaks, and longest streaks.
- Top programming language breakdown with percentage bars.

- Verified project history and testimonials.

2.7 HARDWARE REQUIREMENTS

Client Hardware Compatibility:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (or equivalent ARM Apple Silicon M1/M2/M3).
- **RAM:** Minimum 4 GB RAM (8 GB recommended for running browser devtools and Web3 wallets).
- **Display Resolution:** Optimized across mobile (375px), tablet (768px), laptop (1024px), and desktop (1440px+).
- **Network Interface:** Active internet connection with WebSocket support for real-time chat and transaction notifications.

2.8 SOFTWARE REQUIREMENTS & TECHNOLOGY STACK

Table 2.2: Software Stack & Dependency Matrix

Technology	Package / Tool	Version	Role in Frontend Application
UI Framework	React	^19.0.0	Reactive user interface component hierarchy
Language	TypeScript	^5.5.0	Static typing, interface definitions, API contracts
Build Tool	Vite	^6.4.0	Ultra-fast HMR and optimized production bundling
Styling	Tailwind CSS	^3.4.0	Utility-first styling with custom semantic variables
Client Routing	React Router DOM	^7.0.0	Client-side declarative route management
Web3 Client	Viem	^2.17.0	Ethereum Sepolia RPC client, ABI encoding, contract calls
Icon System	React Icons (Feather)	^5.2.0	Lightweight, accessible SVG icon primitives

HTTP Client	Fetch / Axios Wrappers	Native ES6	Type-safe REST API consumption with JWT handling
Code Quality	TypeScript Compiler	<code>tsc -b</code>	Zero-error build validation and linting

CHAPTER 3: SYSTEM DESIGN & UI/UX ARCHITECTURE

3.1 FRONTEND APPLICATION ARCHITECTURE DIAGRAM

The frontend architecture follows a modular, uni-directional data flow model. Global context providers wrap application routes, feeding state into specialized page views and atomic design components.

FAIRWORK CLIENT PLATFORM

GLOBAL CONTEXT PROVIDERS (APP SHELL)

AuthContext

JWT Token · Roles

PRIMARY APPLICATION PAGES (REACT ROUTER)

LandingPage.tsx

HeroSection · EscrowBlueprint · ProjectCalculator ·
VerifiedRegistry

ProjectPage.tsx

MilestoneTracker · TaskTrigger

ATOMIC DESIGN PRIMITIVES (COMPONENTS/UTILITIES)

Button.tsx

Card.tsx

Badge.tsx

Tabs.tsx

ProgressBar.tsx

Figure 3.1: React 19 Frontend Component Architecture & State Store Hierarchy

The frontend client architecture separates concerns across global application providers (AuthContext, ThemeProvider, ToastProvider), declarative React Router v7 routes, and high-conversion page controllers. Atomic design primitives in components/ui/ provide consistent UX, while projectsApi.ts and web3.ts interface with Express and Ethereum Sepolia.

3.2 USER WORKFLOW STATE DIAGRAMS

3.2.1 Client Project Creation & Milestone Escrow Workflow

This workflow details the sequential path of a client commissioning work, executing legal contracts, and funding the smart contract escrow vault.

COLLABORATION & USER WORKFLOW

WORKFLOW 1: CLIENT ESCROW & RELEASE LIFECYCLE

1. Post Project Brief



2. Review Proposals

WORKFLOW 2: FREELANCER SUBMISSION & GITHUB C

1. Submit Proposal



2. Hired & Sign Contract

WORKFLOW 3: AI DISPUTE MEDIATION & BILATERAL C

1. Open Dispute (Frozen)



2. Request AI Assessment

Figure 3.2: *Client Milestone Escrow Lifecycle & Funding Flowchart*

The client escrow workflow illustrates project posting, proposals evaluation, digital agreement signing, MetaMask USDC allowance approval, and client-gated milestone release payouts.

3.2.2 Freelancer Deliverable Submission & GitHub PR Flow

This workflow illustrates how developers submit work deliverables, attach pull requests, and receive automated CI verification feedback.

COLLABORATION & USER WORKFLOW

WORKFLOW 1: CLIENT ESCROW & RELEASE LIFECYCLE

1. Post Project Brief



2. Review Proposals

WORKFLOW 2: FREELANCER SUBMISSION & GITHUB C

1. Submit Proposal



2. Hired & Sign Contract

WORKFLOW 3: AI DISPUTE MEDIATION & BILATERAL C

1. Open Dispute (Frozen)



2. Request AI Assessment

Figure 3.3: Freelancer Work Submission & GitHub CI Telemetry Sequence

The developer delivery sequence maps code submission through pull requests, parsing CI test runs, displaying verified green badges, and receiving direct non-custodial payouts.

3.2.3 Dispute Mediation & Bilateral Consent Workflow

When milestones diverge from specifications, participants utilize the AI-mediated dispute workflow.

COLLABORATION & USER WORKFLOW

WORKFLOW 1: CLIENT ESCROW & RELEASE LIFECYCLE

1. Post Project Brief



2. Review Proposals

WORKFLOW 2: FREELANCER SUBMISSION & GITHUB C

1. Submit Proposal



2. Hired & Sign Contract

WORKFLOW 3: AI DISPUTE MEDIATION & BILATERAL C

1. Open Dispute (Frozen)



2. Request AI Assessment

Figure 3.4: AI Dispute Mediation Consensus & Bilateral Acceptance Workflow

The dispute workflow outlines vault freezing, automated Gemini AI technical evidence review, the 48-hour consensus countdown window, and arbitrator relay execution upon mutual consent.

3.3 DATA FLOW DIAGRAMS (FRONTEND VIEW)

3.3.1 Level 0 Context Diagram

Illustrates how the frontend client acts as the central orchestrator connecting human users with the backend API and Ethereum blockchain.

3.3.2 Level 1 Client State & Web3 Provider Interaction

Illustrates internal state transitions between React pages, custom API hooks, and Viem blockchain drivers.

DATA FLOW DIAGRAM (LEVEL 1)

Fin



CLIENT

Creates Project · Deposits USDC ·
Releases Funds

1.0 Dep

Client app

2.0 Ever

Active ind



FREELANCER

Submits Code · Attaches GitHub PR ·
Receives Payout

3.0 Reon

Compares

4.0 Fina

Direct non

Figure 3.5: Frontend Level 1 Data Flow Diagram (State & Web3 Interaction)

The frontend Level 1 DFD traces user actions through UI components, REST API consumers, Viem contract callers, and reactive toast notifications.

3.4 WIREFRAME ARCHITECTURE & RESPONSIVE VIEWPORT LAYOUTS

The application layout adapts gracefully across screen dimensions using CSS Grid and Flexbox:

- **Desktop Viewport (\$\ge 1280\text{px}\$):** Implements a balanced 3-column architecture in `ProjectDetailPage`—a 9-column main collaboration workspace paired with a 3-column sticky sidebar presenting quick escrow financial metrics and action buttons.
- **Tablet Viewport (\$768\text{px} - 1024\text{px}\$):** Converts sidebars into stacked cards, ensuring milestone tables remain legible without horizontal overflow.
- **Mobile Viewport (\$375\text{px} - 640\text{px}\$):** Stacks metric cards, collapses tab navigation into a swipeable carousel, and renders modal dialogs as bottom sheets for thumb-friendly interaction.

Table 3.1: Application Route Mapping & Access Authority

URL Path	React Component	Access Role	Primary User Workflow
/	<code>LandingPage</code>	Public	Explore features, interactive escrow simulator, contract registry
/projects	<code>ProjectsPage</code>	Public / Authenticated	Search, filter, and discover open freelance briefs
/projects/new	<code>CreateProjectPage</code>	Client Only	Create technical brief, configure milestone budget schedule
/projects/:id	<code>ProjectDetailPage</code>	Parties & Admin	Collaborate, sign contracts, review CI checks, release funds
/disputes	<code>DisputesPage</code>	Parties & Admin	Inspect AI recommendations, mutual consensus window
/profile/:id	<code>ProfilePage</code>	Public / Authenticated	View developer GitHub stats and on-chain Sepolia reputation

<code>/wallet</code>	<code>WalletPage</code>	Authenticated	Connect MetaMask, view USDC balances, bind EIP-712 address
----------------------	-------------------------	---------------	---

CHAPTER 4: IMPLEMENTATION & CODE METHODOLOGY

4.1 COMPONENT ARCHITECTURE & DESIGN PRIMITIVES (Card, Button, Toast)

To establish visual unity, the design primitives are engineered with TypeScript type safety and accessible DOM structures.

Accessible Button Primitive (Button.tsx):

```
// fairwork-frontend/src/components/ui/Button.tsx
import { forwardRef } from "react"
import { cn } from "@lib/utils"

export interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "secondary" | "outline" | "ghost" | "danger"
  size?: "sm" | "md" | "lg"
  loading?: boolean
  leftIcon?: React.ReactNode
  rightIcon?: React.ReactNode
}

export const Button = forwardRef<HTMLButtonElement, ButtonProps>(({
  className,
  variant = "primary",
  size = "md",
  loading = false,
  leftIcon,
  rightIcon,
  children,
  disabled,
  ...props
}, ref) => {
  const baseStyles = "inline-flex items-center justify-center font-medium rounded-xl
transition-all focus:outline-none focus:ring-2 focus:ring-primary/40 disabled:opacity
-50 disabled:cursor-not-allowed cursor-pointer"
```

```

const variants = {
  primary: "bg-primary hover:bg-primary-hover text-white shadow-sm",
  secondary: "bg-elevated hover:bg-surface-elevated text-foreground border border-b
order",
  outline: "border border-border-strong hover:bg-surface-hover text-foreground",
  ghost: "text-muted hover:text-foreground hover:bg-surface",
  danger: "bg-danger hover:bg-danger/90 text-white shadow-sm"
}

const sizes = {
  sm: "h-8 px-3 text-xs gap-1.5",
  md: "h-10 px-4 text-sm gap-2",
  lg: "h-12 px-6 text-base gap-2.5"
}

return (
  <button
    ref={ref}
    disabled={disabled || loading}
    className={cn(baseStyles, variants[variant], sizes[size], className)}
    {...props}
  >
    {loading ? (
      <span className="h-4 w-4 rounded-full border-2 border-current border-t-transp
arent animate-spin" />
    ) : leftIcon}
    <span>{children}</span>
    {!loading && rightIcon}
  </button>
)
})
Button.displayName = "Button"

```

Context-Driven Toast Alert System (Toast.tsx):

```
// fairwork-frontend/src/components/ui/Toast.tsx

import { createContext, useContext, useState, useCallback, ReactNode } from "react"
import { FiCheckCircle, FiAlertTriangle, FiInfo, FiXCircle, FiX } from "react-icons/fi"

export type ToastTone = "success" | "error" | "warning" | "info"

export interface ToastOptions {
  title: string
  description?: string
  tone?: ToastTone
  duration?: number
}

interface ToastRecord extends ToastOptions {
  id: string
}

interface ToastContextValue {
  toast: (options: ToastOptions) => string
  dismiss: (id: string) => void
}

const ToastContext = createContext<ToastContextValue | null>(null)

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<ToastRecord[]>([])

  const dismiss = useCallback((id: string) => {
    setToasts((prev) => prev.filter((t) => t.id !== id))
  }, [])
```

```

const toast = useCallback((options: ToastOptions) => {
  const id = `toast-${Date.now()}-${Math.random().toString(36).substr(2, 5)}`
  const record: ToastRecord = { ...options, id, tone: options.tone || "info" }
  setToasts((prev) => [...prev, record])

  const duration = options.duration ?? 4500
  if (duration > 0) {
    setTimeout(() => dismiss(id), duration)
  }
  return id
}, [dismiss])

return (
  <ToastContext.Provider value={{ toast, dismiss }}>
    {children}
    <div className="fixed top-5 right-5 z-50 flex flex-col gap-2.5 max-w-sm w-full pointer-events-none">
      {toasts.map((t) => (
        <div
          key={t.id}
          className="pointer-events-auto flex items-start gap-3 rounded-xl border border-surface/95 backdrop-blur-md p-4 shadow-xl animate-in slide-in-from-top-2"
        >
          {t.tone === "success" && <FiCheckCircle className="h-5 w-5 text-success shrink-0" />}
          {t.tone === "error" && <FiXCircle className="h-5 w-5 text-danger shrink-0" />}
          {t.tone === "warning" && <FiAlertTriangle className="h-5 w-5 text-warning shrink-0" />}
          {t.tone === "info" && <FiInfo className="h-5 w-5 text-primary shrink-0" />}
        <div className="flex-1 min-w-0">
          <p className="text-xs font-semibold text-foreground">{t.title}</p>
          {t.description && <p className="text-[11px] text-muted mt-0.5 leading-relaxed">{t.description}</p>}
        </div>
      )

```

```

        </div>
        <button onClick={() => dismiss(t.id)} className="text-subtle hover:text-f
oreground">
            <FiX className="h-4 w-4" />
        </button>
    </div>
    )}
</div>
</ToastContext.Provider>
)
}

export const useToast = () => {
    const ctx = useContext(ToastContext)
    if (!ctx) throw new Error("useToast must be used within ToastProvider")
    return ctx
}

```

4.2 HIGH-CONVERSION LANDING EXPERIENCE (LandingPage.tsx)

LandingPage.tsx integrates the core platform showcase, completely eliminating fake mock personas in favor of verifiable technical reality:

```

// fairwork-frontend/src/pages/LandingPage.tsx
import { LandingHeader } from "@components/landing/LandingHeader"
import { HeroSection } from "@components/landing/HeroSection"
import { EscrowFlowBlueprint } from "@components/landing/EscrowFlowBlueprint"
import { PlatformFeatures } from "@components/landing/PlatformFeatures"
import { ProjectCalculator } from "@components/landing/ProjectCalculator"
import { VerifiedProtocolShowcase } from "@components/landing/VerifiedProtocolShowcase"
import { EscrowAssurance } from "@components/landing/EscrowAssurance"
import { MarketplaceCTA } from "@components/landing/MarketplaceCTA"
import { LandingFooter } from "@components/landing/LandingFooter"

```

```
export function LandingPage() {
  return (
    <div className="min-h-screen bg-base text-foreground scroll-smooth">
      <LandingHeader />
      <main>
        {/* Hero Banner with Universal Search & Interactive Escrow Studio */}
        <HeroSection />

        {/* 4-Stage Automated Milestone Settlement Pipeline */}
        <EscrowFlowBlueprint />

        {/* Core Platform Capabilities: Escrow, AI Mediation, On-Chain Reputation, GitHub CI */}
        <PlatformFeatures />

        {/* Real-time Milestone & Budget Configurator */}
        <ProjectCalculator />

        {/* Verifiable Sepolia Smart Contract Registry & Project Lifecycle */}
        <VerifiedProtocolShowcase />

        {/* Non-Custodial Escrow Security Guardrails */}
        <EscrowAssurance />

        {/* Closing Call-to-Action */}
        <MarketplaceCTA />
      </main>
      <LandingFooter />
    </div>
  )
}
```

4.3 INTERACTIVE MILESTONE & DELIVERABLE HUB (ProjectDetailPage.tsx)

ProjectDetailPage.tsx orchestrates the complete milestone lifecycle, wiring deliverable uploads, GitHub PR checks, and on-chain release triggers:

```
// Excerpt from fairwork-frontend/src/pages/ProjectDetailPage.tsx
export function ProjectDetailPage() {
  const { id } = useParams<{ id: string }>()
  const { user, token } = useAuth()
  const { toast } = useToast()

  const [project, setProject] = useState<ApiProject | null>(null)
  const [deliverables, setDeliverables] = useState<ApiDeliverable[]>([])
  const [ratingModalOpen, setRatingModalOpen] = useState(false)
  const [ciRefreshing, setCiRefreshing] = useState<Record<string, boolean>>({})

  // Refresh GitHub CI Status On-Demand
  const handleRefreshCI = async (deliverableId: string) => {
    if (!token || !project) return
    setCiRefreshing((prev) => ({ ...prev, [deliverableId]: true }))
    try {
      const updatedDel = await refreshDeliverableCI(project.id, deliverableId, token)
      setDeliverables((prev) => prev.map((d) => (d.id === deliverableId ? updatedDel : d)))
      toast({
        title: "CI Checks Refreshed",
        description: `Status updated to ${updatedDel.githubCiStatus.toUpperCase()}`,
        tone: "info"
      })
    } catch (err) {
      toast({ title: "CI Refresh Failed", description: "Unable to reach GitHub API", tone: "error" })
    } finally {
      setCiRefreshing((prev) => ({ ...prev, [deliverableId]: false }))
    }
  }
}
```

```

    }

    // Handle Milestone Approval & Trigger On-Chain Rating on Completion
    const handleMilestoneApprove = async (milestoneId: string) => {
      if (!token || !project) return
      try {
        const updated = await approveMilestone(project.id, milestoneId, token)
        setProject(updated)
        toast({ title: "Milestone Approved", description: "Deliverables approved by client.", tone: "success" })

        // Automatically prompt on-chain rating if project completed
        if (updated.status === "completed") {
          setRatingModalOpen(true)
        }
      } catch (err) {
        toast({ title: "Approval Failed", description: err.message, tone: "error" })
      }
    }

    // Client On-Chain Milestone Payment Release
    const handleReleaseEscrow = async (index: number) => {
      if (!user?.walletAddress) {
        return toast({ title: "Wallet Required", description: "Connect Web3 wallet to release funds", tone: "warning" })
      }
      try {
        const txHash = await releaseEscrowMilestone(project.id, index, user.walletAddress)
        toast({
          title: "Payment Released",
          description: `Milestone funds sent to creator wallet (${txHash.slice(0, 8)}..).`,
          tone: "success"
        })
      }
    }
  }
}

```

```
    })  
    await loadProject()  
  } catch (err) {  
    toast({ title: "Release Failed", description: err.message, tone: "error" })  
  }  
}
```



FairWork

Escrow Pipeline

When Guar

Figure 4.2: FairWork Projects Marketplace with Real-Time Filtering & Milestone Budgets

The discovery marketplace enables freelancers to filter freelance briefs by technical category, budget tier, and deadline urgency, with immediate visual indicators of escrow funding status.

4.4 GITHUB CI STATUS BADGE & REFRESH HOOK IMPLEMENTATION

The GitHub CI badging component renders reactive visual indicators based on pull request test results:

```
{file.githubPrUrl && (  
  <div className="mt-3 flex flex-wrap items-center gap-2.5 rounded-lg border border-b  
order/70 bg-base p-2.5 text-xs">  
    <a  
      href={sanitizeUrl(file.githubPrUrl)}  
      target="_blank"  
      rel="noreferrer"  
      className="font-mono text-primary hover:underline flex items-center gap-1.5 shr  
ink-0"  
    >  
      <FiGithub className="h-3.5 w-3.5" />  
      <span>PR #{file.githubPrUrl.split("/").pop()}</span>  
      <FiExternallink className="h-3 w-3" />  
    </a>  
  
    {file.githubCiStatus === "passed" && (  
      <span className="inline-flex items-center gap-1 text-[11px] font-semibold text-  
emerald-400 bg-emerald-500/10 px-2.5 py-0.5 rounded-full">  
        <FiCheck className="h-3 w-3" /> CI Green ({file.githubCiDetails?.passedChecks  
|| 0}/{file.githubCiDetails?.totalChecks || 0} checks passed)  
      </span>  
    )}  
  
    {file.githubCiStatus === "failed" && (  
      <span className="inline-flex items-center gap-1 text-[11px] font-semibold text-  
danger bg-danger-soft px-2.5 py-0.5 rounded-full">  
        <FiX className="h-3 w-3" /> CI Failing ({file.githubCiDetails?.passedChecks |  
|| 0}/{file.githubCiDetails?.totalChecks || 0} checks passed)  
      </span>  
    )}  
  )}
```

```

    {file.githubCiStatus === "pending" && (
      <span className="inline-flex items-center gap-1 text-[11px] font-semibold text-
warning bg-warning/10 px-2.5 py-0.5 rounded-full">
        <FiClock className="h-3 w-3 animate-spin" /> CI In Progress
      </span>
    )}
    {file.githubCiStatus === "unavailable" && (
      <span className="inline-flex items-center gap-1 text-[11px] text-muted bg-surfa
ce px-2.5 py-0.5 rounded-full">
        <FiShield className="h-3 w-3" /> CI Private / Unavailable
      </span>
    )}

    <Button
      size="sm"
      variant="ghost"
      className="h-6 text-[10px] px-2 ml-auto"
      loading={Boolean(ciRefreshing[file.id])}
      onClick={() => handleRefreshCI(file.id)}
      leftIcon={<FiRefreshCw className={`h-2.5 w-2.5 ${ciRefreshing[file.id] ? "anima
te-spin" : ""}`} />}
    >
      Re-check CI
    </Button>
  </div>
)}

```

4.5 AI DISPUTE MEDIATION PORTAL (DisputesPage.tsx)

DisputesPage.tsx renders the neutral dispute mediation workflow:

```

// Excerpt from fairwork-frontend/src/pages/DisputesPage.tsx
export function DisputesPage() {
  const { id } = useParams<{ id: string }>()
  const { token, user } = useAuth()

```

```
const { toast } = useToast()

const [dispute, setDispute] = useState<ApiDispute | null>(null)
const [evaluating, setEvaluating] = useState(false)
const [accepting, setAccepting] = useState(false)

const handleEvaluateAI = async () => {
  if (!token || !dispute) return
  setEvaluating(true)
  try {
    const updated = await evaluateDisputeWithAI(dispute.id, token)
    setDispute(updated)
    toast({ title: "AI Assessment Complete", description: "Impartial recommendation generated", tone: "success" })
  } catch (err) {
    toast({ title: "Evaluation Failed", description: err.message, tone: "error" })
  } finally {
    setEvaluating(false)
  }
}

const handleAcceptAI = async () => {
  if (!token || !dispute) return
  setAccepting(true)
  try {
    const res = await acceptAiRecommendation(dispute.id, token)
    setDispute(res.dispute)
    toast({ title: "Consent Recorded", description: res.message, tone: "success" })
  } catch (err) {
    toast({ title: "Acceptance Failed", description: err.message, tone: "error" })
  } finally {
    setAccepting(false)
  }
}
```

}

7:53

41.8 KB/s Vo WiFi 35%



fairwork-org.vercel.app/p



Dashboard > Projects > FairWork

FairWork

Web Development

In progress

\$1,000

Created Sep 30, 2026

Deadline: Oct 1, 2026,
5:30 AM

Due in 10
hours

← Back to projects

Overview

Contract

Milestones 3

Files

Activity 1



No Contract Generated Yet

The client will generate the formal legal contract agreement once hiring is complete.



Total
budget

\$1,000



Paid /
Released

\$0

0 of 3 milestones
paid



Ask AI



Unreleased

Figure 4.3: Responsive Project Detail Hub with Multi-Tab Milestone & Contract Management

The mobile-optimized collaboration hub provides quick access to Overview, Contract, Milestones, Files, and Activity tabs, showing live milestone funding progress, deadline countdowns, and action buttons.

4.6 ON-CHAIN RATING MODAL & DUAL STORAGE (OnChainRatingModal.tsx)

OnChainRatingModal.tsx handles client feedback submission, ensuring tolerance against MetaMask rejections:

```
// fairwork-frontend/src/components/feedback/OnChainRatingModal.tsx

export function OnChainRatingModal({ isOpen, onClose, projectId, revieweeId, reviewee
Name, revieweeAddress, onSuccess }: OnChainRatingModalProps) {

  const { token } = useAuth()
  const { toast } = useToast()

  const [rating, setRating] = useState(5)
  const [comment, setComment] = useState("")
  const [submitOnChain, setSubmitOnChain] = useState(Boolean(revieweeAddress))
  const [submitting, setSubmitting] = useState(false)

  if (!isOpen) return null

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!token) return
    setSubmitting(true)
    let onChainSucceeded = false
    let minedTxHash = ""

    try {
      // 1. Submit on-chain rating to Sepolia ReputationContract if requested
      if (submitOnChain && revieweeAddress) {
        try {
          minedTxHash = await submitOnChainRating(projectId, revieweeAddress, rating,
comment)
          onChainSucceeded = true
        }
      }
    }
  }
}
```

```

        toast({
          title: "On-Chain Rating Confirmed",
          description: `Published to Sepolia ReputationContract (${minedTxHash.slice(0, 10)}...).`,
          tone: "success",
        })
      } catch (onChainErr: any) {
        console.warn("On-chain rating failed or was cancelled:", onChainErr)
        toast({
          title: "On-Chain Transaction Declined",
          description: "MetaMask transaction declined. Proceeding to save platform review...",
          tone: "warning",
        })
      }
    }
  }

  // 2. Persist review to MongoDB for instant UI display
  const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000/api"
  const res = await fetch(`${API_URL}/reviews`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
    body: JSON.stringify({ projectId, revieweeId, rating, comment, onChainTxHash: minedTxHash || undefined }),
  })

  if (!res.ok) throw new Error("Failed to save review to platform database")

  toast({
    title: "Review Submitted",
    description: onChainSucceeded ? "Written to Sepolia blockchain and saved to profile!" : "Review saved successfully!",
    tone: "success",
  })

```

```

    })

    onSuccess?.()
    onClose()
  } catch (err: any) {
    toast({ title: "Review Failed", description: err.message, tone: "error" })
  } finally {
    setSubmitting(false)
  }
}

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/70 backdrop-blur-sm animate-in fade-in">
    <div className="relative w-full max-w-lg bg-surface border border-border/80 rounded-2xl shadow-2xl p-6">
      {/* Rating stars, comment textarea, Sepolia toggle, and Submit button */}
    </div>
  </div>
)
}

```

4.7 WEB3 WALLET CONNECTION & EIP-712 INTEGRATION (web3.ts)

web3.ts provides typed wrappers for client-side EVM interactions using Viem:

```

// fairwork-frontend/src/services/web3.ts
import { createWalletClient, custom, parseUnits } from "viem"
import { sepolia } from "viem/chains"

export const reputationAddress = "0xfa25823ccf7343fd7fa20a785d996331e66674" as const
export const escrowAddress = "0xc0d1b74a30a82d6fb846e446758a8c2ff391376c" as const

export async function submitOnChainRating(

```

```

    projectId: string,
    reviewee: string,
    score: number,
    comment: string
  ): Promise<string> {
    if (typeof window === "undefined" || !window.ethereum) {
      throw new Error("No Ethereum wallet found. Please install MetaMask.")
    }

    const walletClient = createWalletClient({
      chain: sepolia,
      transport: custom(window.ethereum)
    })

    const [account] = await walletClient.requestAddresses()

    const txHash = await walletClient.writeContract({
      address: reputationAddress,
      abi: REPUTATION_ABI,
      functionName: "submitRating",
      args: [projectId, reviewee as `0x${string}`, score, comment],
      account
    })

    return txHash
  }

```

CHAPTER 5: TESTING, USABILITY & PERFORMANCE VERIFICATION

5.1 TYPESCRIPT COMPILATION & TYPE SAFETY VERIFICATION

The frontend codebase is enforced under strict TypeScript compiler rules (`tsc -b`). All API responses, state payloads, and component props are strictly typed to eliminate runtime undefined errors.

```
$ npm run build

> fairwork@2.0.0 build
> tsc -b && vite build

vite v6.4.3 building for production...
transforming...
✓ 1417 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  1.51 kB | gzip:  0.76 kB
dist/assets/index-Bybl7KDn.css  129.69 kB | gzip: 18.63 kB
dist/assets/ProjectDetailPage-90aF5jbR.js  78.01 kB | gzip: 18.02 kB
dist/assets/LandingPage-T54vYtcJ.js       56.18 kB | gzip: 12.43 kB
dist/assets/DisputesPage-YURXFY8f.js      9.83 kB | gzip:  3.17 kB
dist/assets/vendor-viem-khBKC_I3.js      290.66 kB | gzip: 88.94 kB

✓ built in 6.47s
$ npm run lint

> fairwork@2.0.0 lint
> tsc --noEmit

# Exited with code 0 (Zero TypeScript compilation or lint errors)
```

5.2 RESPONSIVE VIEWPORT & CROSS-DEVICE COMPATIBILITY TESTING

The UI was evaluated across four standard device form factors:

1. **iPhone 14 / Mobile (390px × 844px):**

- Sticky desktop sidebars collapse into stacked sections.
- Project detail tabs render as a smooth horizontal scrolling carousel.
- Modals adapt to touch-friendly bottom sheets.

1. **iPad Air / Tablet (820px × 1180px):**

- 2-column grid layout for deliverables and milestone rows.
- Header navigation collapses into an animated mobile drawer.

1. **MacBook Pro / Laptop (1440px × 900px):**

- Full 3-column desktop layout with sticky financial metrics sidebar.

1. **Desktop Monitor (1920px × 1080px):**

- Content max-width constrained to `max-w-7xl` with centered optical margins.

5.3 USER INTERACTION & FORM VALIDATION TESTING

Table 5.1: Client Form Validation & Security Test Matrix

Form / Flow	Input / Test Case	Validation Rule	UI Feedback Behavior	Result
Project Creation	Budget set to <code>-\$500</code>	Budget must be <code>\$ > 0\$</code>	Red border, error text: <code>"Budget must be positive"</code>	PASS
Milestone Decomposition	Sum of milestones <code>\$\neq\$</code> total	<code>\$\$\sum m_i == \text{budget}\$\$\$</code>	Disables submit, highlights allocation error	PASS
PR Attachment	Malformed URL entered	Valid GitHub PR regex	Inline alert: <code>"Invalid GitHub PR URL format"</code>	PASS
Dispute Submission	Empty dispute reason	Non-empty text required	Highlights textarea, blocks submit	PASS
Rating Submission	Rating score <code>\$= 0\$</code>	Score <code>\$\in [1, 5]\$\$\$</code>	Star selector clamps between 1 and 5	PASS

5.4 WEB3 METAMASK TRANSACTION STATE HANDLING & FEEDBACK

Table 5.2: Web3 Transaction Error Recovery Verification

Scenario ID	User / Network Action	Wallet State	Frontend Recovery Action
W3-01	User clicks "Fund Escrow"	MetaMask Extension Missing	Prompts modal: "Please install MetaMask"
W3-02	User rejects transaction	User rejected in MetaMask (Code 4001)	Catches rejection, displays amber warning toast
W3-03	Wrong Network connected	Connected to Mainnet instead of Sepolia	Prompts automatic wallet_switchEthereumChain
W3-04	On-chain rating declined	User declines rating gas transaction	Gracefully falls back to saving MongoDB review

5.5 LIGHTHOUSE PERFORMANCE, ACCESSIBILITY & SEO AUDITING

Table 5.3: Google Lighthouse Performance & Accessibility Scores

Category Metric	Score	Target	Key Architectural Factors
Performance	94 / 100	≥ 90	Code-split Vite chunking, lazy image loading, SVG icons
Accessibility (a11y)	98 / 100	≥ 95	High-contrast text tokens, semantic button/input labels
Best Practices	100 / 100	≥ 95	Zero console errors, modern HTTPS headers, sanitized URLs
SEO	100 / 100	≥ 95	Valid semantic hierarchy, structured meta tags, breadcrumbs

CHAPTER 6: CONCLUSION & FUTURE ENHANCEMENTS

6.1 SUMMARY OF INDIVIDUAL ACHIEVEMENTS

The frontend design and implementation executed by **VIGNESH V** delivered an enterprise-grade user experience for FairWork:

1. **Design System & Component Primitives:** Created an accessible, responsive Tailwind CSS design system with custom variable tokens and dark/light mode support.
2. **Authentic Discovery Experience:** Purged all fake mock personas and replaced them with the authentic PlatformFeatures and VerifiedProtocolShowcase components displaying real Sepolia contracts.
3. **Collaborative Project Hub:** Architected ProjectDetailPage.tsx, uniting milestone tracking, deliverable files, and legal agreements.
4. **Objective GitHub CI Badging:** Built the CI review interface that parses pull request test runs and displays verified test status badges without removing client approval authority.
5. **AI Dispute Mediation Interface:** Implemented DisputesPage.tsx, displaying Gemini AI recommendations and managing the 48-hour mutual consent window.
6. **Zero-Error Type Safety:** Achieved 100% clean compilation on Vite and TypeScript (tsc -b), ensuring exceptional client stability.

6.2 KEY TAKEAWAYS & DESIGN REFLECTIONS

- **Human-Centric Web3 UX:** Users should never be forced to parse raw hexadecimal hashes; visualizing state transitions through clear status pills and milestone timelines bridges the adoption gap.
- **Resilient Non-Blocking Fallbacks:** When interacting with blockchain wallets, user interfaces must anticipate transaction rejections and provide non-blocking fallbacks.
- **Authenticity Inspires Trust:** Replacing fake stock personas with verified smart contract registries and live pull request checks significantly enhances user confidence.

6.3 FUTURE WORK & FRONTEND ROADMAP

1. **Progressive Web App (PWA) Support:** Implement service worker caching and offline manifest configurations for mobile home-screen installation.
2. **WalletConnect v2 Integration:** Support mobile hardware wallets and QR code pairing via WalletConnect.
3. **Live Milestone Video Previews:** Support inline deliverable video streams and 3D canvas renderers directly within the Files tab.
4. **Decentralized Chat via XMTP:** Transition off-chain Socket.IO messaging to the Extensible Message Transport Protocol (XMTP) for end-to-end encrypted Web3 wallet-to-wallet chat.

APPENDIX: ROUTE DEFINITIONS & API SCHEMA TYPINGS

A.1 Core Application Route Map

```
export const appRoutes = [  
  { path: "/", element: <LandingPage /> },  
  { path: "/projects", element: <ProjectsPage /> },  
  { path: "/projects/new", element: <CreateProjectPage /> },  
  { path: "/projects/:id", element: <ProjectDetailPage /> },  
  { path: "/disputes", element: <DisputesPage /> },  
  { path: "/profile/:id", element: <ProfilePage /> },  
  { path: "/wallet", element: <WalletPage /> },  
  { path: "/chat", element: <ChatPage /> }  
]
```

A.2 Typed Project & Deliverable API Interfaces

```
export interface ApiDeliverable {  
  id: string  
  filename: string  
  url: string  
  size: number  
  mimeType?: string  
  uploadedAt: string  
  uploadedByName?: string  
  milestoneId: string  
  submissionNotes?: string  
  githubPrUrl?: string  
  githubCiStatus?: "passed" | "failed" | "pending" | "unavailable"  
  githubCiDetails?: {  
    totalChecks: number  
    passedChecks: number  
    failedChecks: number  
    pendingChecks: number  
  }  
}
```

```
}
```

```
}
```

REFERENCES

1. React Documentation. (2024). *React 19: Design Patterns and Server Components*. <https://react.dev>
2. Vite Documentation. (2024). *Vite: Next Generation Frontend Tooling*. <https://vitejs.dev>
3. Tailwind CSS. (2024). *Tailwind CSS v3.4: Utility-First CSS Framework*. <https://tailwindcss.com>
4. Viem. (2024). *Viem Documentation: TypeScript Interface for Ethereum*. <https://viem.sh>
5. W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG22/>
6. Nielsen, J. (1994). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group.
7. Google Developers. (2024). *Lighthouse: Automated Auditing Tool for Web Page Quality*. <https://developer.chrome.com/docs/lighthouse/>
8. GitHub Inc. (2024). *GitHub Checks API Overview*. <https://docs.github.com/en/rest/checks>
9. Ethereum Foundation. (2018). *EIP-712: Typed Structured Data Hashing and Signing*. <https://eips.ethereum.org/EIPS/eip-712>
10. Material Design / Linear App Design Essays. (2023). *Precision Typography and High-Density Web UI Systems*.